Journal of Machine Learning Research XX (2026) 1–9 Submitted MM/YY; Published MM/YY

# fastEmbedR: Native CPU, Metal, and CUDA Implementations of UMAP and openTSNE-Style t-SNE for R

**Moussa Kassim**[1,2] Moussa.Kassim@icgeb.org
**Martin Ocharo**[1,2] Martin.Ocharo@icgeb.org
**Alessia Vignoli**[3,4] vignoli@cerm.unifi.it
**Leonardo Tenori**[3,4] tenori@cerm.unifi.it
**Stefano Cacciatore**[1,2] stefano.cacciatore@icgeb.org
[1] *Bioinformatics Unit, International Center for Genetic Engineering and Biotechnology, Cape Town 7925, South Africa*
[2] *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*
[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy*
[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

**Editor:** To be assigned

## Abstract

The most widely used accelerated implementations of t-distributed stochastic neighbor embedding (t-SNE) and Uniform Manifold Approximation and Projection (UMAP) are fragmented across R, Python, C++, and accelerator-specific libraries. We present `fastEmbedR`, an R package that implements complete UMAP and openTSNE-style t-SNE workflows with native CPU, Apple Metal, and NVIDIA CUDA execution. Here, "openTSNE-style" denotes algorithmic lineage, not compatibility with Python openTSNE. The package accepts either a data matrix or reusable nearest-neighbor indices and distances, constructs UMAP graphs or t-SNE affinities internally, and executes optimization without calling Python. Its numerical core uses compact float32 and int32 storage, while GPU paths retain major working arrays on device. Backend-selectable randomized PCA supplies t-SNE initialization without implying a universal speed advantage, and explicit landmark functions support reference embedding and projection of new observations. In three-seed HPC benchmarks across 11 image, cytometry, metabolomics, and single-cell datasets, nine datasets supported matched full-pipeline comparisons. Median speedups (dataset-level interquartile ranges [IQRs]) were 1.81 (0.97–2.75) and 54.2 (15.9–64.9) for CPU/CUDA openTSNE versus Rtsne, and 1.31 (1.28–1.79) and 63.5 (31.8–66.3) for fuzzy UMAP versus uwot fast SGD. Median trustworthiness differences were +0.039, +0.041, −0.0012, and −0.0012, respectively. `fastEmbedR` therefore makes accelerated embeddings directly available to R users through one coherent, inspectable API without concealing runtime or quality trade-offs.

**Keywords:** dimensionality reduction, t-SNE, UMAP, R software, GPU computing, Metal, CUDA

## 1 Introduction

t-SNE and UMAP are standard tools for high-dimensional biological and machine-learning data (van der Maaten and Hinton, 2008; McInnes et al., 2018; Kobak and Berens, 2019). Barnes–Hut t-SNE, FIt-SNE, openTSNE, and GPU UMAP improved their scalability (van der Maaten, 2014; Linderman et al., 2019; Poličar et al., 2024; Nolet et al., 2020), but R users still encounter separate package interfaces, external executables, Python environments, and backend-specific data transfers. Moreover, a complete run includes nearest-neighbor search, affinity or graph construction, initialization, and iterative optimization; accelerating only one stage may not improve the user-visible call.

`fastEmbedR` addresses this software gap with one R interface backed by native CPU, Apple Metal, and NVIDIA CUDA implementations. It is not a wrapper around Python openTSNE or RAPIDS cuML. The package owns KNN-to-affinity/graph conversion, initialization, sparse schedules, optimization state, landmark projection, return objects, timing metadata, and backend validation. Optional platform libraries provide narrowly defined primitives such as CUDA search, truncated SVD, or FFTs. The resulting public API supports both complete matrix-input workflows and direct reuse of precomputed KNN matrices.

## 2 Software Architecture and Implementations

### 2.1 R interface and computational boundary

The matrix-input functions `opentsne()` and `umap()` accept ordinary R matrices. They also accept float32 matrices from the `float` package. The KNN-entry functions, `opentsne_knn()` and `umap_knn()`, accept reusable integer nearest-neighbor indices and floating-point distances. Backend selection is explicit: `backend="cpu"`, `"metal"`, or `"cuda"`. An unavailable GPU request fails rather than silently running on CPU.

For matrix input, CPU nearest-neighbor search uses package-resident hierarchical navigable small world (HNSW), distilled from permissively licensed FAISS code (Johnson et al., 2021; Douze et al., 2024); Metal uses exact or recall-tuned inverted-file flat (IVF-Flat); CUDA links FAISS GPU exact or cuVS IVF-Flat search and can retain KNN on-device. The exported `pca()` uses randomized singular value decomposition (RSVD) (Halko et al., 2011). On CPU it uses `fastPLS::pca(method="rsvd")` when a compatible fastPLS installation is available and otherwise uses the package-native RSVD path; Metal uses float32 block-subspace TSVD with Metal Performance Shaders (MPS), while CUDA uses RAFT TSVD. Setting `opentsne_init=TRUE` produces the small-scale PCA initialization used by t-SNE (Kobak and Berens, 2019; Cacciatore, 2026).

### 2.2 Native UMAP

Fuzzy UMAP estimates per-row $\rho_i$ and $\sigma_i$, forms directed memberships, and combines reciprocal edges as $w_{ij} + w_{ji} - w_{ij}w_{ji}$ (McInnes et al., 2018). The explicit `graph_mode="binary"` alternative replaces these memberships with unit weights on the symmetric KNN union. It is an adjacency-only sensitivity mode for data where nearest-neighbor identity is trusted more than distance calibration, not standard or necessarily faster UMAP. Both modes use the package's low-dimensional attraction/repulsion, negative sampling, learning-rate decay, and size-aware epoch policy.

The graph is stored once in compressed sparse row (CSR) form with int32 offsets/indices and float32 weights/schedules. CPU code parallelizes bandwidth estimation, graph construction, sparse initialization products, and edge updates. Metal uploads the schedule once and keeps optimizer state in unified GPU memory. CUDA can consume resident KNN, prepare the graph on-device, and execute the package-native atomic optimizer. `uwot` is a behavioral benchmark, not a dependency (Melville, 2026).

### 2.3 Native openTSNE-style t-SNE

Here, "openTSNE-style" denotes algorithmic lineage, not Python software compatibility. `fastEmbedR` implements the published t-SNE KL objective, perplexity-matched sparse affinities, early-exaggeration and normal phases, adaptive gains and momentum, FIt-SNE interpolation/FFT repulsion, and fixed-reference transformation (van der Maaten and Hinton, 2008; Linderman et al., 2019; Poličar et al., 2021, 2024; Belkina et al., 2019). It neither calls nor ports Python openTSNE and does not claim compatibility with its API, objects, defaults, trajectories, or coordinates.

`opentsne_knn()` owns affinity construction and optimization. Package code supplies the R/KNN APIs, nearest-neighbor policy, RSVD initialization, automatic rules, float32 sparse storage, and kernels for CPU, Metal, and CUDA. CPU reuses FFT plans, grids, and worker scratch. Metal runs grid scatter, package-native FFT convolution, interpolation, attraction, and updates; its validated Stockham transform follows permissively licensed Apple GPU design ideas (Bergach, 2026). CUDA retains optimizer and cuFFT state on-device and groups iterations with CUDA Graph execution. The sparse-attraction/interpolated-repulsion design follows FIt-SNE and t-SNE-CUDA without invoking their executables (Linderman et al., 2019; Chan et al., 2018).

### 2.4 Precision and scalable transforms

Working data, distances, weights, coordinates, gradients, gains, and workspaces use float32; graph indices use int32. Float32 R input avoids an initial double conversion and receives a float32 layout, although total process memory also includes R objects, indices, FFT grids, and library workspaces. `landmark_tsne()` and `landmark_umap()` embed a reference subset, interpolate remaining observations, apply a local affine correction, and optionally refine projected points. The t-SNE transform holds reference coordinates fixed as in Poličar et al. (2021); the result records the approximation and each timing component. Reusable initialization and optimizer-only entry points are described in the supplement.

## 3 Evaluation

We evaluated 11 datasets spanning 873 to 5,220,347 observations and 11 to 16,384 variables. Each method was run with seeds 4, 17, and 42; $k = 30$ or perplexity 30; four requested CPU threads or one CUDA device; and the per-method configuration reported in the supplement. Competing methods on a machine used the same R environment, compiler toolchain, and thread controls. Cross-package comparisons report total public-function time because reference implementations expose different boundaries; KNN and initialization are not subtracted. Labels are used only for post hoc metrics (Venna and Kaski, 2001;

Rousseeuw, 1987). Figures 1 and 2 report t-SNE and UMAP runtime separately; Figure 3 shows the six requested MNIST embeddings. The supplement reports every selected method–dataset–backend row, including failures and unavailable runs, a complete numerical quality table, and one six-method embedding figure per dataset.

On MNIST70k, median total times were 69.56 and 1.94 seconds for CPU/CUDA openTSNE, 125.83 seconds for Rtsne, and 116.49 seconds for FIt-SNE. Fuzzy UMAP required 50.04 and 0.99 seconds on CPU/CUDA, binary UMAP 113.97 and 1.05 seconds, and uwot fast SGD 65.34 seconds. Across nine matched datasets, median speedups (IQRs) were 1.81 (0.97–2.75) and 54.2 (15.9–64.9) for CPU/CUDA openTSNE versus Rtsne, and 1.31 (1.28–1.79) and 63.5 (31.8–66.3) for fuzzy UMAP versus uwot fast SGD. Corresponding median trustworthiness differences were $+0.039$, $+0.041$, $-0.0012$, and $-0.0012$; median Preserve@30 differences for fuzzy UMAP were $-0.0013$ and $-0.0024$. CPU peak-RSS ratios were 0.81 versus Rtsne and 0.63 versus uwot. Every comparison requires a successful within-dataset pair. FlowRepository (fastEmbedR CUDA only) and ImageNet (fastEmbedR CPU only) are feasibility results and support no comparative claim.

Correctness was evaluated at matched mathematical boundaries. On a fixed 2,000-point MNIST test, t-SNE affinity edge Jaccard and Pearson agreement with Python openTSNE were 1.000; fuzzy UMAP graph edge Jaccard and weight Pearson agreement with uwot were 1.000 and 0.99999. With identical KNN input and initialization across three seeds, median common-affinity t-SNE KL was 1.923, 1.899, and 1.762 for CPU, Metal, and CUDA, versus 1.766 for Python openTSNE. Median full-embedding UMAP Procrustes correlations were 0.995 (CPU–CUDA), 0.957 (CPU–Metal), and 0.994 (CPU–uwot). Final-neighborhood and per-seed results are reported in the supplement; they support behavioral agreement without claiming coordinate or bitwise identity.

## 4 Discussion and Availability

`fastEmbedR` exposes t-SNE and UMAP on multicore CPU, Metal, and CUDA with matrix/KNN input, float32 storage, PCA initialization, and transforms. Platform libraries provide primitives; graph/affinity construction and optimization are package code. Native PCA enables backend integration and device residency but is not uniformly faster (supplement).

Fixed seeds reproduce sampling and random streams, but asynchronous atomic order precludes bitwise GPU reproducibility. Landmark functions interpolate and affinely correct an explicit reference subset, optionally refine against fixed coordinates, and report component timings.

Binary UMAP is a sensitivity mode, not a CPU acceleration: across ten matched datasets, median binary/fuzzy runtime ratios (IQRs) were 2.24 (1.89–2.58) on CPU and 1.06 (1.03–1.09) on CUDA. Unit weights schedule every retained edge maximally, whereas fuzzy weights sample weak edges less often. Quality varied on small datasets (supplement); fuzzy mode is the standard comparison and binary results remain separate.

Landmarking accelerated median CPU but not CUDA runs; medians and IQRs are supplementary. `fastEmbedR` is MIT licensed at `https://github.com/tkcaccia/fastEmbedR`. Source is frozen at commit `58e39d5` (full identity in the supplement); an archival DOI will accompany the final release.

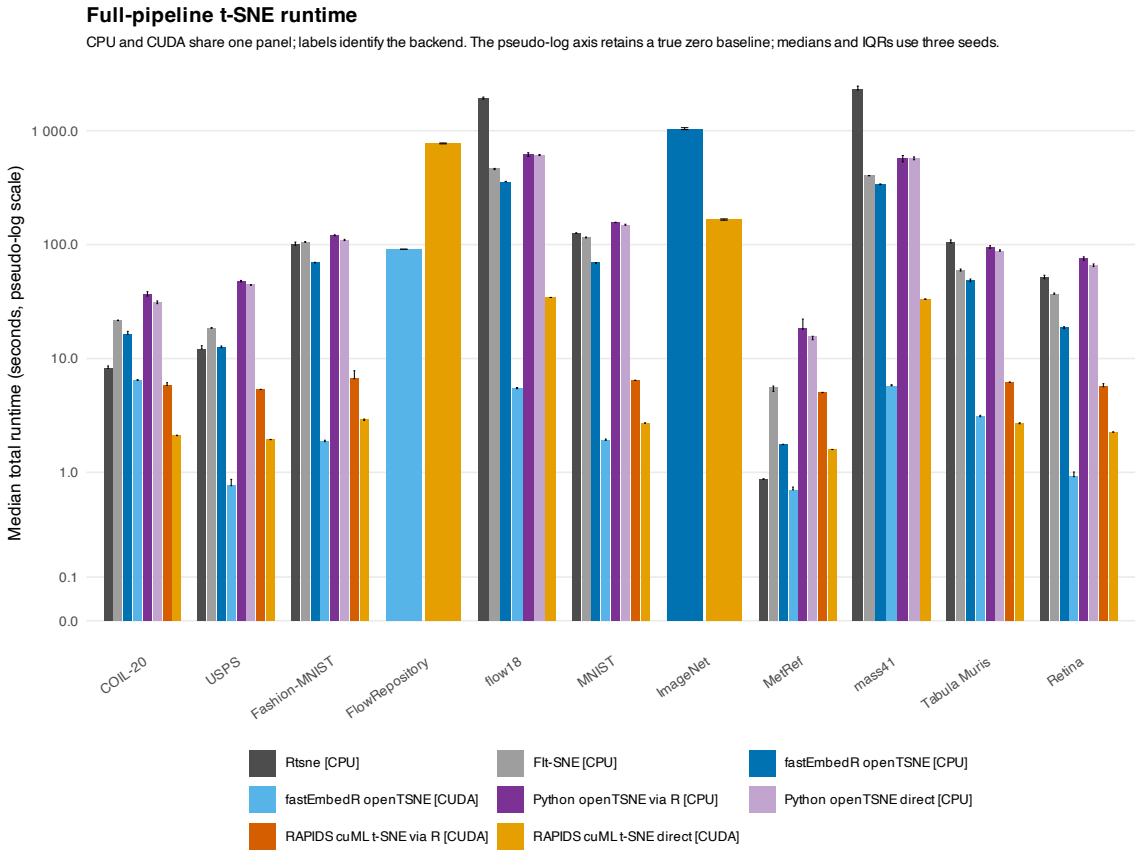


Figure 1: Total full-pipeline runtime for the available t-SNE implementations. The single panel includes CPU and CUDA results, identified explicitly in the legend, for Rtsne, FIt-SNE, `fastEmbedR` openTSNE, Python openTSNE, and RAPIDS cuML t-SNE. Gray denotes established R references, blue denotes `fastEmbedR`, purple denotes Python openTSNE, and orange denotes RAPIDS; shades distinguish backend or timing interface. The continuous pseudo-log runtime axis retains a true zero baseline. Bars are medians over seeds 4, 17, and 42; error bars span the interquartile range. Direct-Python and R-mediated calls remain separate because their timing boundaries differ. Missing or failed rows remain in the supplementary result files. Unmatched FlowRepository and ImageNet rows are descriptive only and excluded from comparative summaries.

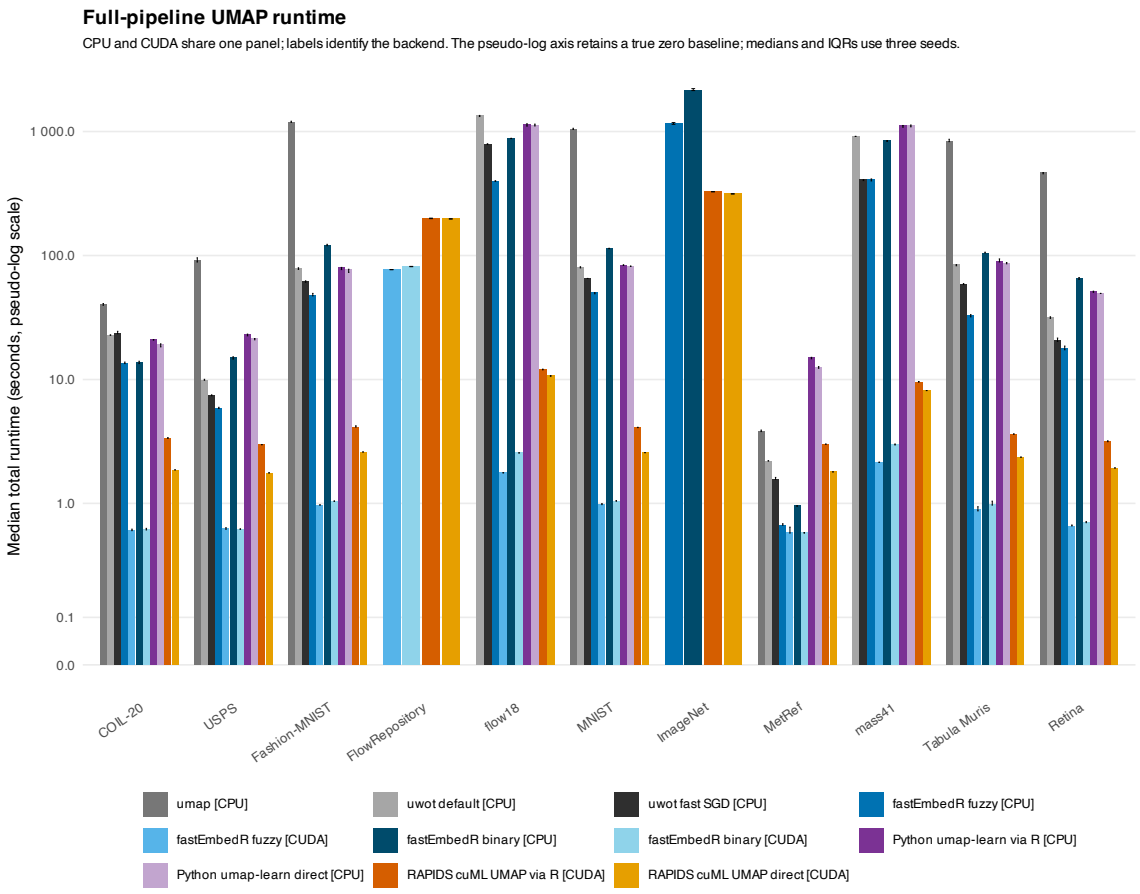


Figure 2: Total full-pipeline runtime for the available UMAP implementations. The single panel combines CPU and CUDA results, identified in the legend, for the R `umap` package, uwot default and fast-SGD modes, `fastEmbedR` fuzzy and binary modes, Python umap-learn, and RAPIDS cuML UMAP. Colors and timing conventions follow Figure 1; the pseudo-log runtime axis likewise retains a true zero baseline. Binary `fastEmbedR` results are an explicitly labeled adjacency-only approximation; fuzzy mode is the standard comparison. Unmatched FlowRepository and ImageNet rows are descriptive only.

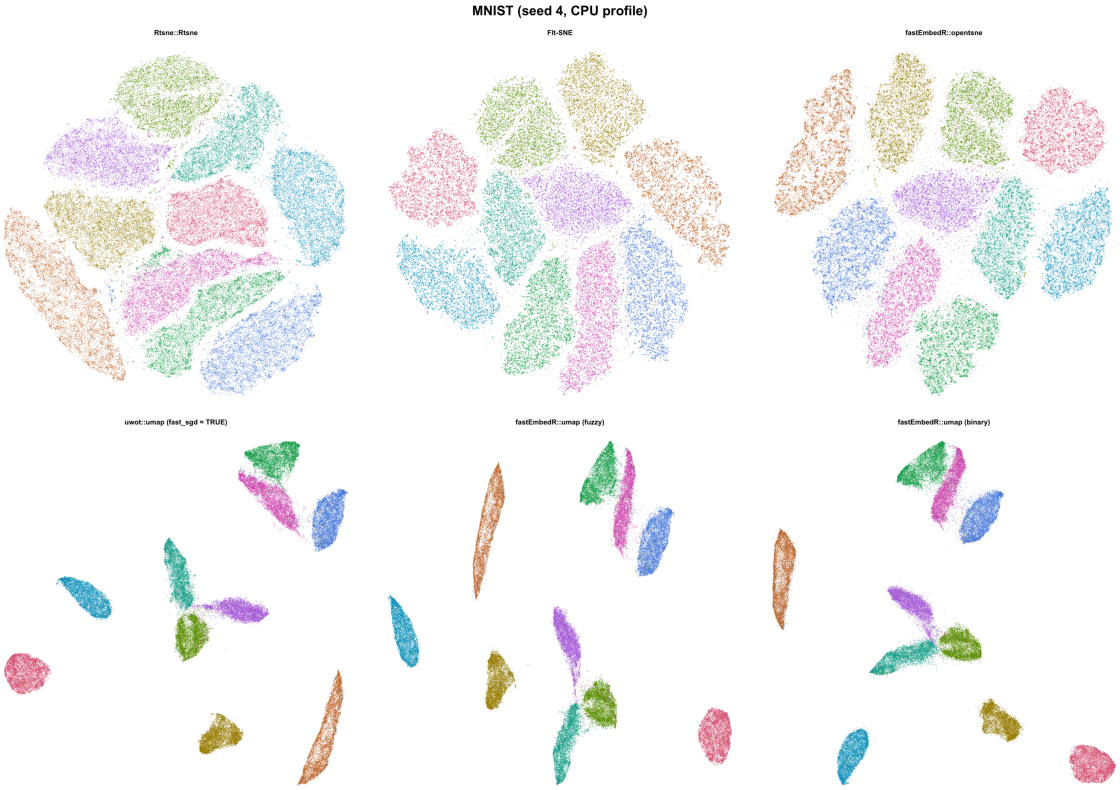


Figure 3: MNIST70k embeddings for Rtsne, FIt-SNE, `fastEmbedR` openTSNE, uwot UMAP, and the `fastEmbedR` fuzzy and binary UMAP modes (seed 4, four-thread CPU profile). Labels determine point color only and were not used during fitting. Plots contain points only, without axes or boxes. Equivalent six-method panels for every dataset are supplied in the supplement.

## References

Anna C. Belkina, Christopher O. Ciccolella, Rina Anno, Richard Halpert, Josef Spidlen, and Jennifer E. Snyder-Cappione. Automated optimized parameters for t-distributed stochastic neighbor embedding improve visualization and analysis of large datasets. *Nature Communications*, 10:5415, 2019. doi: 10.1038/s41467-019-13055-y.

Mohamed Amine Bergach. AppleSiliconFFT. Software repository, 2026. URL `https://github.com/aminems/AppleSiliconFFT`.

Stefano Cacciatore. fastPLS: Accelerated partial least squares and randomized matrix decompositions for r. R package, 2026. URL `https://github.com/tkcaccia/fastPLS`.

David M. Chan, Roshan Rao, Forrest Huang, and John F. Canny. t-SNE-CUDA: GPU-accelerated t-SNE and its applications to modern data. *arXiv preprint arXiv:1807.11824*, 2018. doi: 10.48550/arXiv.1807.11824.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. doi: 10.48550/arXiv.2401.08281.

Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

Dmitry Kobak and Philipp Berens. The art of using t-SNE for single-cell transcriptomics. *Nature Communications*, 10:5416, 2019. doi: 10.1038/s41467-019-13056-x.

George C. Linderman, Manas Rachh, Jeremy G. Hoskins, Stefan Steinerberger, and Yuval Kluger. Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data. *Nature Methods*, 16:243–245, 2019. doi: 10.1038/s41592-018-0308-4.

Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. doi: 10.48550/arXiv.1802.03426.

James Melville. uwot: The uniform manifold approximation and projection method for dimensionality reduction. R package, 2026. URL `https://github.com/jlmelville/uwot`.

Corey J. Nolet, Victor Lafargue, Edward Raff, Thejaswi Nanditale, Tim Oates, John Zedlewski, and Joshua Patterson. Bringing UMAP closer to the speed of light with GPU acceleration. *arXiv preprint arXiv:2008.00325*, 2020. doi: 10.48550/arXiv.2008.00325.

Pavlin G. Poličar, Martin Stražar, and Blaž Zupan. Embedding to reference t-SNE space addresses batch effects in single-cell classification. *Machine Learning*, 110:1211–1231, 2021. doi: 10.1007/s10994-021-06043-1.

Pavlin G. Poličar, Martin Stražar, and Blaž Zupan. openTSNE: A modular python library for t-SNE dimensionality reduction and embedding. *Journal of Statistical Software*, 109 (3):1–30, 2024. doi: 10.18637/jss.v109.i03.

Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. doi: 10.1016/0377-0427(87)90125-7.

Laurens van der Maaten. Accelerating t-SNE using tree-based algorithms. *Journal of Machine Learning Research*, 15:3221–3245, 2014. URL `https://www.jmlr.org/papers/v15/vandermaaten14a.html`.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. URL `https://www.jmlr.org/papers/v9/vandermaaten08a.html`.

Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In *Artificial Neural Networks – ICANN 2001*, pages 485–491. Springer, 2001. doi: 10.1007/3-540-44668-0_68.